

Constraint logic programming for scheduling and planning

J Lever, M Wallace and B Richards

This paper provides an introduction to finite-domain constraint logic programming (CLP) and its application to problems in scheduling and planning. The fundamentals of CLP are covered and recent developments and trends in the field are indicated. Some current limitations are identified, and areas of research that may contribute to addressing these limitations are suggested.

1. Introduction

Constraint logic programming (CLP) languages are extensions of logic programming systems such as PROLOG with constraint-handling facilities. There have been two lines of development — systems such as the CLP(X) scheme [1] and PROLOG III [2] in which constraints are handled by specific black-box solvers, and systems such as CHIP [3] and cc(FD) [4], in which constraints over finite-domain variables are handled through constraint propagation. Recently, new techniques such as generalised propagation [5] and constraint handling rules [6] have been developed, providing propagation in a more general context than finite-domains and allowing constraint-solving to be applied to userdefined predicates rather than only to a restricted set of system-defined primitives.

This paper concentrates on finite-domain CLP, while including descriptions of generalised propagation and constraint-handling rules, as these techniques are of most relevance for scheduling and planning. In fact, the development of the original CHIP system at ECRC, Munich, was motivated by the weakness of logic programming when faced with combinatorial search and optimisation problems found in the context of scheduling and planning, and many papers exist reporting the application of CHIP to such problems (see, for example, Dincbas et al [7, 8]). Industrial sectors in which CLP has had impact include transportation, telecommunications and manufacturing. Some commercially available CLP languages are ICL's Decision Power, Cosytex's CHIP4, Bull's Charme and Siemens' SNI-Prolog. Generalised propagation and constraint-handling rules are available alongside finite-domain constraint propagation in the ECLiPSe CLP language, which has been developed at ECRC as a successor to CHIP. ECLiPSe is in use at a rapidly growing number of academic sites throughout the world. Although it is not currently commercially available,

a licence has been granted to BT Laboratories for research purposes.

The paper is organised as follows. After an introduction to the basics of CLP programming in section 2, the application of CLP to scheduling applications is described in section 3. Finally, in section 4 some current limitations of CLP are indicated and some proposals are made through which to address them. The proposals made in this section are to be followed up in a new project, RESPONSE, at IC-Parc. At the moment, CLP is one of a range of approaches that should be considered for scheduling and planning problems. In particular, an important issue is the integration of techniques from other approaches such as operations research and approximation algorithms with constraint handling. It is possible that a future synthesis of techniques will lead to increases in performance and solution quality over all current approaches.

2. Constraint logic programming — the basics

This section presents the basic notions of CLP and indicates how it can be used to solve constraint-satisfaction problems. Finding a solution to a constraint-satisfaction problem requires choosing values for variables so as to satisfy the constraints of the problem. In the context of scheduling, variables typically represent the times at which tasks are started or ended and the resources used to perform a task. Of course, most scheduling problems also include notions of cost, and the way in which CLP makes provision for finding not just any solution but a good or optimal solution to the problem is described. It should be noted that in this paper variables will be represented by upper case as in X or Start, while constants are represented by lower case as in bill, machine1.

2.1 Constraints

The simplest form of constraint found in CLP systems is implicit in the notion of a finite-domain variable. A finite-domain is a finite set of values, and a finite-domain variable is simply a variable that is constrained to take values from an associated finite-domain. For example, in CLP:

`Start :: 1..10`

is used to express the constraint that the start time represented by the variable `Start` must take a value between 1 and 10. Similarly:

`Engineer :: [bill,john,mary]`

can be used to express the constraint that the job whose engineer variable is `Engineer` must be done by either `bill`, `john`, or `mary`¹.

Linear inequalities and disequalities are also provided as constraints in CLP systems. Some examples are:

`End-Start>=5`

`Start1<Start2`

`Engineer1 ne Engineer2`

CLP systems typically also include some global constraints. Global constraints express restrictions that could otherwise be imposed by a set of other constraints, but where formulation at the global level allows efficiency to be gained. For example, the constraint:

`alldifferent([X1, X2, X3, X4, X5])`

states that none of the variables $X_1, \dots, X_5$ can take the same value.

2.2 Propagation

The evaluation of constraints over finite-domains in CLP is as follows:

- when a constraint is first imposed, the domains of its variables are checked for evidently inconsistent values, and any such values are removed,
- if all combinations of the remaining values satisfy the constraint, the constraint is solved,
- otherwise the constraint is 'put to sleep', i.e. its evaluation is suspended until some future point at which it will be reconsidered.

As an example, suppose that the variables `Start1` and `Start2` each have domain 1..10, and that the constraint `Start1<Start2` is imposed. Then it can be seen that the variable `Start2` cannot take the value 1, as this leaves no value for `Start1` satisfying the constraint. Similarly, `Start1` cannot take the value 10. These values are removed from their respective domains, leaving it in the form:

`Start1(1..9)<Start2(2..10)`

In constraint-solving terms, the constraint has been made 'arc-consistent' [9]. However, it is not solved as it is possible to assign values from the domains which violate the constraint, for example `Start1=9`, `Start2=2`. So the constraint is put to sleep.

Suppose that, after this first constraint is imposed, subsequently a second constraint `Start(1..9)<5` is imposed. Arc-consistency applied to the second constraint reduces the domain of `Start1` to 6..9, and this constraint is now solved as it is satisfied by all remaining values of `Start1`. The reduction to the domain of `Start1` causes the sleeping constraint to wake, as it may now be possible to extract some further information (in the form of domain reductions) from it. In fact, arc-consistency now leaves the first constraint in the form:

`Start1(6..9)<Start2(7..10)`

What is seen in this example is reductions in the domains of variables being propagated to other constraints involving the same variables, causing their consistency to be re-checked, and possibly generating further domain reductions. Domains can be reduced not only through the action of constraints, but also by explicit assignment as in `Start1=8`. Propagation alone does not always detect inconsistency among the current constraints, and often does not instantiate the problem variables. To ensure consistency and to generate a schedule in which all problem variables are instantiated, it is necessary to augment propagation with search.

2.3 Constrained search

The key to the success of CLP in tackling combinatorial search problems involving constraints lies in the combination of propagation and search. The search component consists of the assignment of values of variables from their domains, often called labelling. Each element in the domains represents an alternative choice, and creates a branch in the search-tree. By imposing the constraints involved in the problem before commencing search, the size of the domains can be reduced through the action of consistency checking and propagation. Furthermore, as the search proceeds, the effects of assignments made to variables are propagated to the domains of the, as yet unlabelled, variables, pruning the possibilities that remain open. Thus, propagation acts to reduce the branching in the search-tree by eliminating paths that do not lead to solutions in advance.

¹ In some CLP systems the elements of finite domains are restricted to integers, so that a trivial mapping from constraints to integers can be required.

2.4 Cost minimisation

The requirement for an optimal solution rather than just any solution is commonly dealt with in CLP by applying the Branch-and-Bound technique to the search component of the algorithm. A cost function is required, and as variables are labelled during search, the lower bound of the cost function can rise as a result of the choices made. When a solution is found, its cost is placed as an upper bound on the cost of any future solution. Thus, when the cost of a partial labelling exceeds the current bound, backtracking is forced as the partial labelling could not be extended to a solution that improves on that already found.

2.5 Generalised propagation, constraint-handling rules, and user-defined constraints

The above description summarises the facilities of a CLP system such as the original ECRC CHIP. Recent work at ECRC in the context of the CHIP ESPRIT project has given rise to the current ECLiPSe system, which extends CHIP along a number of dimensions. A significant restriction of CHIP was that constraint solving was only available through a restricted set of system-defined primitives. The techniques of generalised propagation and constraint-handling rules (CHRs) incorporated in the ECLiPSe allow user-defined relations to act as constraints. In addition, the behaviour of such constraints is specified at a high level, and it is therefore easy to experiment with different forms and strengths of constraint-solving and so uncover the most efficient strategy for the particular problem in hand. At a more procedural level, ECLiPSe includes sophisticated delay mechanisms which can also be used to obtain constraintsolving behaviour from user-defined relations.

3. Solving scheduling problems in CLP

Scheduling problems are typically solved in CLP using the model of constrained search sketched in the previous section, i.e. impose the problem constraints and label the problem variables. This section points out some features of scheduling problems that complicate matters, namely disjunctive constraints and the existence of similar solutions of identical cost, and discusses how these complications can be overcome. Also indicated is scope for heuristics in solving problems of sufficient scale to defeat exhaustive search techniques.

3.1 Disjunctive constraints

Contention for resources, as in the case where two tasks require the same machine, introduces the need for disjunctive [7]. In this case the disjunctive constraint is

expressed as a rule with two clauses, one for each constraint¹.

```
contention(Task1, Task2) ←
    start(Task1) + duration(Task1) < start(Task2)
contention(Task1, Task2) ←
    start(Task2) + duration(Task2) < start(Task1)
```

However, for many problems such an approach introduces too many choice points and yields an unsatisfactory performance [10].

Disjunctive constraints can be used completely passively, by simply checking them for consistency. In this case the constraint is not used to prune the domains of any problem variables, it merely causes failure if, at any point in the search, all the disjunctions are inconsistent. However, a more active use is possible if the system can detect when all but one alternative is excluded, and then impose the remaining alternative as an active constraint. In this case the individual disjunctions are checked for consistency, instead of the whole disjunctive constraint. When only one consistent disjunction remains, it is imposed as an active constraint, and can be used for pruning the domains of the remaining variables.

The simplest way to achieve this behaviour is to introduce an auxiliary variable with a binary domain [10]. In fact it has proved effective to include the auxiliary variable in the labelling routine. This gives the effect of setting the constraint at a choice point as above, but in this case the labelling routine can select the variable for labelling at the best point in the search.

Although one binary variable suffices for a choice point with just two choices, the solution with auxiliary variables becomes more complicated and less efficient for processing when there are multiple choices. Multiple choices arise naturally, e.g. when there are multiple tasks in contention for one, or even several, resources. To avoid an increasing number of auxiliary variables it is possible to obtain a similar constraint behaviour by using CHRs. As an example, the rule for the contention problem is:

```
contention(Task1, Task2) ⇔
    start(Task1) + duration(Task1) = start(Task2) |
    start(Task1) = start(Task2) + duration(Task2)
contention(Task1, Task2) ⇔
    start(Task2) + duration(Task2) = start(Task1) |
    start(Task2) ≤ start(Task1) + duration(Task1)
```

¹ This syntax for rules is not the CHIP compiler syntax.

These rules yield the following behaviour. If contention (Task1,Task2) is a current goal then the system continually checks the conditions:

```

start(Task1) + duration(Task1) > start(Task2)
and
start(Task2) + duration(Task2) > start(Task1)

```

As soon as one of the conditions is entailed by the scheduling choices made so far (say the first one, for example) the system immediately replaces the goal contention (Task1,Task2) with the body of the rule (in this case $\text{start}(\text{Task1}) = \text{start}(\text{Task2}) + \text{duration}(\text{Task2})$).

Of course the number and complexity of such CHRs also increases with the number of contending tasks. However, the power of multiheaded CHRs can be used to avoid adding each pair of contending tasks as an explicit constraint. If any two tasks are assigned to the same machine, they are in contention. Therefore, only two CHRs are needed for all contending tasks:

```

assign(T1,M,ST1),assign(T2,M,ST1) ⇔
  start(T1)+duration(T1)=start(T2) |
  start(T1)=start(T2)+duration(T2)
assign(T1,M,ST1),assign(T2,M,ST2) ⇔
  start(T2)+duration(T2)=start(T1) |
  start(T2)=start(T1)+duration(T1)

```

In this case it is the CHR processor that detects all the contending tasks and enforces the constraint on each of them.

The amount of propagation invested in a disjunctive constraint has a significant effect on overall performance. In Dincbas et al [7] various ways of using disjunctive constraints for a scheduling problem were investigated. The constraints were used for lookahead — a powerful form of propagation — for forward checking, which corresponds approximately to the weak form of propagation proposed by Chamard and Fischler [10], and as choice points. For the particular scheduling application at hand, the use of disjunctive constraints as choice points proved to be more efficient.

Generalised propagation [5] allows the disjunctive constraints to be expressed as ordinary Prolog rules (as in the first example above). When the constraint is invoked as a goal, it is then possible to specify how it should be used — for propagation, or for search. A way of experimenting with different amounts of propagation is offered by approximate generalised propagation [11]. The advantage is that different amounts of propagation can be achieved by modifying a single parameter. The change has no effect on program semantics and is a small syntactic change, which makes experimentation, program support and maintenance much easier.

3.2 Eliminating symmetries

Many scheduling problems have isomorphic classes of solutions, i.e. classes of solutions of identical cost that differ only by exchanging one particular resource value for another throughout the schedule. It is desirable to avoid repeated generation of isomorphic configurations during the search for an optimal solution, as the size of the search space will then be drastically reduced. A problem which had been solved with CHIP [8] in which such configurations occur was solved even more efficiently by a new approach which used dynamically generated nogood assertions to prune branches of the search tree according to this criteria [12]. The idea is based on the concept of no-good environments introduced by assumption-based truth maintenance systems [13]. The approach is complementary to constraint propagation - propagation is performed before and during search, whilst no-goods are extracted after the search has failed. Both propagation and no-goods are used to prune the search tree subsequently. Theoretically, such information could also be extracted by propagation, but the difficulty is in choosing what propagation information is worth extracting. In the no-good approach there is no need to make such a decision, since the information is extracted as a side-effect of a failed search that had to be done anyway.

Experiments with this approach have been made using CHIP augmented with no-goods. It was used to tackle a real application concerned with shift timetabling. The improved backtrack behaviour enabled the whole search space to be explored and an optimum timetable to be found.

3.3 Heuristics for large scheduling problems

For many large practical problems, it is not possible to search all possible schedules, even using sophisticated constraint techniques to prune impossible schedules from the search tree. Consequently, much of the research on scheduling is about heuristics to focus the search on promising parts of the search tree, without any attempt to cover all possibilities. This section will concentrate on heuristics that guide both the order in which variables are labelled, and the order in which the values for labelling the selected variables are chosen.

Variable selection

Many scheduling problems have parts that are relatively easy to satisfy and other parts which are much harder. The hard parts of a schedule are termed 'bottlenecks'. When searching for a solution, if the bottlenecks are left till last, then they may be unsolvable because of certain choices made earlier which could have easily been changed. Therefore, it is best to try to deal with the bottlenecks first,

since each choice is important. The other parts of the schedule can then be fitted around the bottlenecks. This is often known as the ‘first-fail’ principle.

To identify bottlenecks the concept of variable ‘tightness’ was introduced in Fox et al [14]. A tight variable is one that eliminates lots of possible solutions — specifically ‘the tightness of a variable is the probability that an assignment, consistent with all the problem constraints that do not involve the variable, does not result in a solution’. The first-fail principle requires the search routine to label tighter variables first. However, variable tightness is not a fixed measure. A certain choice for one variable may leave very few options for another variable. Essentially the influence of variables on each other is dictated by the problem constraints. Therefore another measure, which helps select those variables to be labelled next, is based on the constraints - this is called ‘constraint tightness’ [13].

Value selection

Given that a certain variable is to be labelled, the next requirement is to label it in a way that allows the maximum freedom in labelling the remaining variables. A direct measure of the total reduction in the domains of the remaining task variables may not be very useful if the real difficulty is with one or two other particular variables. A better way to control the labelling of variables is to measure ‘variable contention’ which records the extent to which a number of constraints on the variable conflict with each other. Two variables constrained by a common set of constraints should be labelled in such a way as to minimise this contention.

Preference propagation

In order to use the measures of tightness and contention outlined above, some technique of estimating them is required. The techniques described in Lepape and Smith [15] and Sadeh and Fox [16] are based on constraint propagation. In this context, the purpose of propagation is not to eliminate impossible values from the domains of variables, but rather to identify degrees of freedom for variables and sources of contention.

Implementation in CLP

CLP offers the appropriate control to support variable and value selection. Moreover, the modification required to generalised propagation to support preference propagation is not dramatic. The requirement is to retreat all the alternatives and combine them into a set of preferences. Generalised propagation already provides the facility to extract information from the set of alternative answers to a goal - the difference is merely in the operations that build the result from the answers as they are extracted. The

general problem of assessing tightness and contention by preference propagation is by no means a solved problem. However, it seems likely that as new forms of preference propagation are devised, it will be possible to incorporate them into the CLP propagation framework.

4. Research horizons

In this section some current and future research directions are discussed, including techniques for predictive and reactive scheduling, constrained approximation, and use of CLP to solve planning problems.

4.1 Predictive and reactive scheduling

The requirement for both predictive and reactive scheduling is a feature of many, if not most, real scheduling applications. A technique which is possible using finite domains is to produce, as output from the predictive scheduler, not just the chosen start time for each task, but a finite set of possible start times.

The reactive scheduler associates a variable with each task, as usual, and labels it with the start time when it starts. As long as every task starts on time (i.e. at the start time chosen by the predictive scheduler), then no constraint checking is necessary, since that has already been done during the predictive schedule.

However, as soon as a task is labelled with an unpredicted start time, all the constraints on this variable are woken. Propagation on these constraints may remove the predicted start time from the domains of other tasks. The constraints on these tasks are then also woken. Thus, the effects of an unexpected event are allowed to ripple through the constraint network, waking up all constraints that are affected - but no other constraints.

A typical example of such a reactive behaviour occurs in train timetabling. When one train is late it is necessary to modify the timetables for trains which must wait for it, but not for other trains. The challenge is to identify the affected part of the railway network and reschedule it, with the aim of achieving minimal disruption.

The tools necessary for such a reactive scheduling mechanism in CLP are provided by CHRs. For each task there is a rule - for example:

```
react(T) ⇔ start(T)=25 | true.  
react(T) ⇔ start(T) ne 25 | C1(T,T2), C2(T,T3)...
```

where 25 is the start time for T chosen by the predictive scheduler, C1(T,T2), C2(T,T3),... are the constraints on task T. This rule does nothing until start(T) either becomes equal to 25, or becomes constrained not to equal 25. In the second

case all the constraints on T are awoken, and used during the subsequent propagation and labelling. This approach is still in its infancy, and much further work will be required to make it mature and usable in practice.

4.2 Optimisation by constrained approximation

Constraint logic programming supports search over a solution space structured into a tree, some of whose leaves are feasible solutions. The constraints allow some of the branches, whose leaves include no feasible solutions, to be pruned away in advance during the search. Many problems require not just a feasible solution but an optimal one, assuming some function associating a cost with each solution. To find the optimum, some kind of enumeration of the feasible solutions is required. The enumeration efficiency can be improved by pruning away in advance branches whose leaves include no optimal feasible solutions (Branch-and-Bound, described earlier), or by other well-known techniques such as cutting planes, or dynamic programming. Unfortunately, for large problems, even the pruned search tree is often too large to explore in realistic timescales; thus the enumeration method is computationally impracticable.

Approximation algorithms

In case it is impracticable to find the actual optimum by enumeration, approximation algorithms can be used to find, with a high probability, a good solution, by exploring only a part of the solution space. For many hard search problems, such as the travelling salesman problem, assembly-line sequencing, and scheduling, approximation algorithms have been used very successfully [17-19].

Such methods are often viewed as an alternative to constrained search. In fact, constraints can be used with approximation algorithms in exactly the same way they are used with enumeration algorithms - the choice of approximation or enumeration is independent of the use of constraints. Experiments carried out show that constraints bring the same benefits of simplicity and efficiency when used with approximation algorithms as with enumeration algorithms [20].

Intuitively, approximation algorithms, such as hillclimbing, simulated annealing and genetic algorithms, work when solutions which are similar have a broadly similar cost — the cost function is, in a loose sense, continuous. When there is no relationship between the costs of similar solutions, then there is no reason why an approximation method should perform any better than standard backtracking. In this case, both approximation algorithms and standard backtracking just pick one solution after another, recording the best solution found so far.

The notion of 'similarity' between solutions is captured by operators defined for the problem at hand. For hill-climbing and simulated annealing these are 'neighbourhood' operators which produce new solutions similar to a given one; for genetic algorithms they are 'recombination' operators which produce a new solution which is an appropriate mixture of two existing ones.

The need for constrained approximation

Unconstrained approximation algorithms make no distinction between feasible solutions and others. Therefore, the elimination of unfeasible solutions has to be achieved by a kind of trick. Violated constraints have an associated cost. For example, in a hill-climbing algorithm for the n -queens problem [19] the cost of a solution is simply the number of pairs of queens which can take each other — the number of violated constraints. For problems which involve both hard constraints (e.g. two tasks cannot run on one machine at the same time), and soft constraints (e.g. job J must be completed by date D or as soon as possible afterwards), it is important to ensure that solutions violating the hard constraints are eliminated. This is achieved by associating with them a very high cost, so that solutions with a reasonably low overall cost are ones that do not violate hard constraints.

The immediate consequence of this approach is that 'similar' solutions, as defined in terms of the operators described above, no longer necessarily have a similar cost. Thus the graph of solutions against their costs is no longer broadly continuous, but full of spikes. When applied to problems whose solution space has such an irregular cost function, the intuitive basis for the effectiveness of the approximation algorithm is no longer satisfied.

Constrained approximation avoids this difficulty. It works by constraining the search for an initial solution and constraining the operators that produce new (similar) solutions from old ones. The consequence is that the spikes in the graph of solutions against their costs are smoothed out. It should be noticed that the unfeasible solutions cannot simply be dropped from the solution space. This would leave gaps in the solution space in place of spikes. It could prevent certain 'good' solutions being recombined during the genetic algorithm, and for hillclimbing and simulated annealing it could leave some solutions without any neighbours at all. Rather than the constraints being used passively, therefore, they are used actively to guide the neighbourhood or recombination operator to a feasible solution.

Experiments and conclusions

These experiments have focused on the travelling salesman problem (TSP), using a constrained genetic algorithm. Three problems were studied — a small TSP and

a larger TSP without specific constraints, and the larger TSP with a number of specify constraints of different kinds (e.g. a constraint on the distance travelled from a certain location to the subsequent one). For a simple TSP problem without constraints, Branch-and-Bound is the effective technique. However, a larger problem without constraints is better solved using the genetic algorithm. The larger problem was then modified by the inclusion of constraints. These, declaratively specified, constraints were used by the constrained genetic algorithm during generation of an initial set of solutions, and during the recombination of pairs of descendants. The effect of constraints during recombination is to enforce some variables to take certain values and to preclude other variables from taking the value proposed by the recombination operator. The latter variables are subsequently labelled under the constraints in the usual CLP manner. The constrained TSP problem was too large to be solved by enumeration techniques, but very good solutions were found by the constrained genetic algorithm. These experiments are described in more detail in Kuchenoff [20].

These experiments and the work on the theoretical foundations of constrained approximation are still in progress. Based on early results there is optimism that the technique will be effective for a broad range of problems. It is believed that the incorporation of constrained approximation algorithms into CLP, as an alternative to Branch-and-Bound, will enable the technology to be scaled up to larger real-world problems. Another likely source of techniques to increase the scalability of CLP solutions is operations research (OR) algorithms. A recent paper has reported very successful results obtained in the context of the 10?10 job-shop scheduling problem through the incorporation of OR techniques with CLP [21].

4.3 Constraint logic programming for planning

So far, the discussion has focused on scheduling problems. Planning problems can be distinguished from scheduling problems according to the following criteria: in scheduling, the set of tasks to be achieved or attempted is fixed, while in planning, achieving a task can introduce a new task so that the set of tasks to be attempted cannot be specified in advance. Planning problems are therefore a superset of scheduling problems and inherit their complications, whilst also requiring a higher level of reasoning. Constraint-solving is therefore a useful tool for tackling planning problems, and the generic planning architecture *parcPLAN* [22], developed at IC-Parc, has used constraint logic programming to build a generic shell with capabilities in planning, scheduling and resource allocation. So far, *parcPLAN* has been applied to blocksworld planning - the traditional domain for AI planners — and to a flight allocation problem provided by British Airways. *parcPLAN* will be further developed on the RESPONSE grant to

incorporate a number of the extensions described above in connection with scheduling.

5. Conclusions

This paper has presented the basics of constraint logic programming and shown its relevance for solving scheduling and planning problems. Features of scheduling problems which present difficulty in general have been described, and the manner and extent in which these difficulties can be handled currently within CLP has been explained. One of the most important results has been the ease with which it is possible to implement and experiment with various different algorithms and heuristics in CLP - particularly through the use of generalised propagation and constraint-handling rules as are provided in the ECLiPSe system. An important horizon is the integration of constraint-solving with approximation algorithms, and the extensibility of the CLP paradigm may well lead to new ways of combining these techniques to deliver enhanced performance.

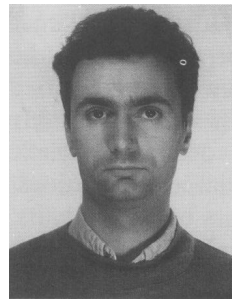
6. Acknowledgements

Thanks are due to the researchers of IC-Parc, the CHIC user group for lots of great discussions.

References

- 1 Jaffar J and Lassex J L: 'Constraint logic programming', in Proceedings of the Fourteenth ACM Symposium on Principles of Programming Languages (POPL'87), Munich (January 1987).
- 2 Colmerauer F: 'An introduction to Prolog III', CACM, 33, pp 69—90 (July 1990).
- 3 Dincbas M, Van Hentenryck P, Simonis H, Aggoun A, Graf T and Berthier F: 'The constraint logic programming language chip', in Proceedings of the International Conference on Fifth Generation Computer Systems (FGCS'88), pp 693—702, Tokyo, Japan (December 1988).
- 4 Van Hentenryck P, Simonis H and Dincbas M: 'Constraint satisfaction using constraint logic programming', Artificial Intelligence, 58 (1992).
- 5 Le Provost T and Wallace G: 'Generalised constraint propagation over the CLP scheme', J of Logic Programming, 16 (1993).
- 6 Fruhwirth T: 'Constraint simplification rules', Technical Report ECRC18, ECRC (July 1992).
- 7 Dincbas M, Simonis H and Van Hentenryck P: 'Solving large combinatorial problems in logic programming', J of Logic Programming, 8 (1990).
- 8 Dincbas M, Simonis H and Van Hentenryck P: 'Solving a cuttingstock problem in constraint programming', in Fifth International Conference on Logic Programming (ICLP'88), Seattle, USA, MIT Press (August 1988).
- 9 Mackworth A K: 'Consistency in networks of relations', Artificial Intelligence, 8, pp 99-118 (1977).
- 10 Chamard A and Fischler A: 'Applying CHIP to a complex scheduling program', Technical Report D3.1.2, Dassault Aviation, CHIC report (1993).

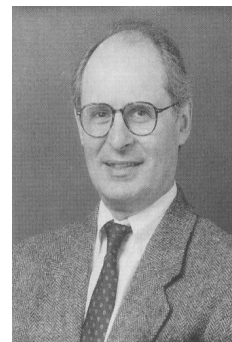
- 11 Le Provost T: 'Approximation in the framework of generalised propagation', Technical report, ECRC, presented at the CLP workshop, FGCS'92 (1993).
- 12 Maruyama F, Minoda Y, Sawada S and Takizawa Y: 'Constraint satisfaction and optimisation using nogood justifications', in Proc 2nd Pacific Rim Conf on AI (1992).
- 13 de Kleer J: 'An assumption-based TMS', *Artificial Intelligence*, **28**, pp 127-162 (1986).
- 14 Fox M S, Sadeh N and Baycan C: 'Constrained heuristic search', in Proc IJCAI, pp 309—316 (1989).
- 15 Lepape C and Smith S F: 'Management of temporal constraints for factory scheduling', in IFIP Working Conf on Temporal Aspects in Information Systems (1987).
- 16 Sadeh N and Fox M S: 'Preference propagation in temporal/capacity constraint graphs', Technical Report CMU-RI-TR-89-2, Robotics Institute, Carnegie Mellon University (1989).
- 17 Muhlenbein H: "Parallel genetic algorithms and combinatorial optimization", *SIAM J on Optimization* (1992).
- 18 Chew T L, David J M, Nguyen L and Tourbier Y: 'Le car sequencing problem revisité: analyse d'une utilisation du recuit simulé', Technical report, RENAULT, Service Systèmes Experts (1992).
- 19 Minton S, Johnston M D, Philips A B and Laird P: 'Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems', *Artificial Intelligence*, **58** (1992).
- 20 Kuchenhoff V: 'Novel search and constraints — an integration', Technical report, ECRC, CHIC deliverable (1992).
- 21 Caseau Y and Laburthe F: 'Improved CLP scheduling with task intervals', in Eleventh International Conference on Logic Programming (ICL'94), MIT Press (1994).
- 22 Lever J M and Richards B: 'ParcPLAN: a planning architecture with parallel actions, resources and constraints', in Proc ISMIS 94 (October 1994).



Dr Jonathan Lever is a Research Fellow at IC-Parc, and has been working as a researcher at Imperial College for eight years. He completed a PhD in Computational Logic in 1991. The work described in this paper is part of Project RESPONSE, a collaborative program with BT. His current research interests are in constraint programming techniques and their application to planning and scheduling. His recent work has been on the design and implementation of the generic planning architecture, parcPLAN.



Dr Mark Wallace has been with ICL for some 15 years, during which he completed a PhD at Southampton University, and then spent a decade at the European Computer Industry Research Centre (ECRC), first working on knowledge bases, and for the last three years taking over leadership of ECRC's constraint reasoning project. He is currently seconded from ICL to ICParc, Imperial College where he is an Honorary Principal Research Fellow. He has research interests in constraint logic programming and constraint databases. His aim is to support the development of simple efficient correct programs for handling complex industrial decisions both in databases and stand-alone programs.



Professor Barry Richards is Director of the Centre for Planning and Resource Control (IC-Parc) at Imperial College, where he also holds an established Chair of Computing Science. He received a BA degree from the University of Notre Dame (Indiana) in 1964, and a PhD from the University of Cambridge in 1972. He joined the Department of Computing at Imperial College in May 1989, having previously been Professor of Cognitive Science in the University of Edinburgh and Director of the Centre for Cognitive Science. His current research interests are in planning and scheduling, temporal reasoning, and constraint solving, covering both their theoretical foundations and practical implementations.